Java Team Project: Warehouse Robot Fleet Simulation

Duration: 4 weeks Team size: 4–5 students Interface: Console-based

1. Project Overview

Design and build a simulation of a small automated warehouse. Robots move through a warehouse, collect items from pickup locations, and deliver them to designated drop-off points. The system must demonstrate how robot behavior, warehouse conditions, and work assignments change over time.

Your team is responsible for selecting the architecture, classes, relationships, algorithms, and data structures. No particular design or implementation is prescribed.

2. Required Functionality (Minimum Viable Project)

• Warehouse layout: Represent a small warehouse with navigable spaces, fixed obstacles, pickup points, and delivery points. Display the warehouse state in a readable console format.

• Robot fleet: Simulate at least three robots, each with an identifiable position, battery level, and current status (such as idle, moving, or completing a task).

• Tasks: Create pickup-and-delivery tasks, assign tasks to available robots, and track tasks from pending to completed.

• Movement: Advance the simulation in discrete steps. Robots must move within warehouse boundaries and must not move through obstacles. A simple valid route-selection strategy is sufficient; optimal pathfinding is not required.

• Battery behavior: Robot movement consumes battery. When battery is insufficient, the robot must stop or take a safe, clearly documented recovery action. Automatic charging is not required.

• Simulation controls: Allow the user to initialize or reset a scenario, add tasks, advance the simulation, and inspect robot and task status.

• Errors and edge cases: Handle invalid commands, unreachable destinations, unavailable robots, and other foreseeable failures without crashing.

3. Scope Boundaries

A console application with a small, predefined warehouse is sufficient. A graphical interface, networking, real-time concurrency, database, complex route optimization, and sophisticated multi-robot traffic management are not required. Teams may use simple, documented rules when multiple robots compete for space or tasks. Prioritize a coherent, reliable simulation over extra features.

4. Object-Oriented Programming Expectations

Apply the Java OOP concepts taught in class where they are appropriate: encapsulation, abstraction, inheritance, polymorphism, abstract classes/interfaces, constructors, method overriding/overloading, exceptions, collections, and separation into multiple source files. Do not introduce artificial inheritance or interfaces merely to check a box; justify design decisions.

5. Collaboration and GitHub

Each team must use a shared GitHub repository, divide work into meaningful parallel contributions, work on branches, use pull requests and peer review, and maintain a functional integrated version. Every member must contribute to design, implementation, testing, and presentation. Teams choose their own work allocation and integration strategy.

6. Four-Week Schedule

Week	Expected progress
1 - Design & foundation	Agree on scope, Git workflow, initial UML, interfaces, and a running basic warehouse simulation.
2 - Core implementation	Implement robots, task assignment, movement, and initial end-to-end workflows.
3 - Integration & behavior	Complete battery behavior, integrate subsystems, and handle operational edge cases.
4 - Test & present	Resolve defects, test edge cases, finalize documentation/UML, and deliver a live team demo.

7. Deliverables

Submit: (1) complete working source code, (2) GitHub repository with meaningful collaboration history, (3) UML class diagram, (4) brief architecture and design-decision document, (5) README with build/run instructions, (6) evidence of tests and edge cases, and (7) a live demo in which every team member participates.

8. Acceptance Demonstration

The final demonstration must show a warehouse scenario with at least three robots, multiple tasks, at least one successful pickup-and-delivery sequence, visible simulation progress, battery consumption, and appropriate handling of at least two failure or edge cases.

9. Optional Enhancements (Not Required for Full Credit)

Possible extensions include smarter pathfinding, collision avoidance, charging stations, task prioritization, saved scenarios, or performance statistics. Optional work must not replace completion of required functionality.

10. Grading

Functional correctness: 30%
OOP design and code quality: 25%
Git collaboration and teamwork: 15%
Testing and error handling: 15%
UML and documentation: 10%
Live demonstration: 5%